



UNIVERSIDAD
COMPLUTENSE
MADRID

‘Creación de una Base de Datos’

Pablo Blas Hernández

1. Diseño e-r

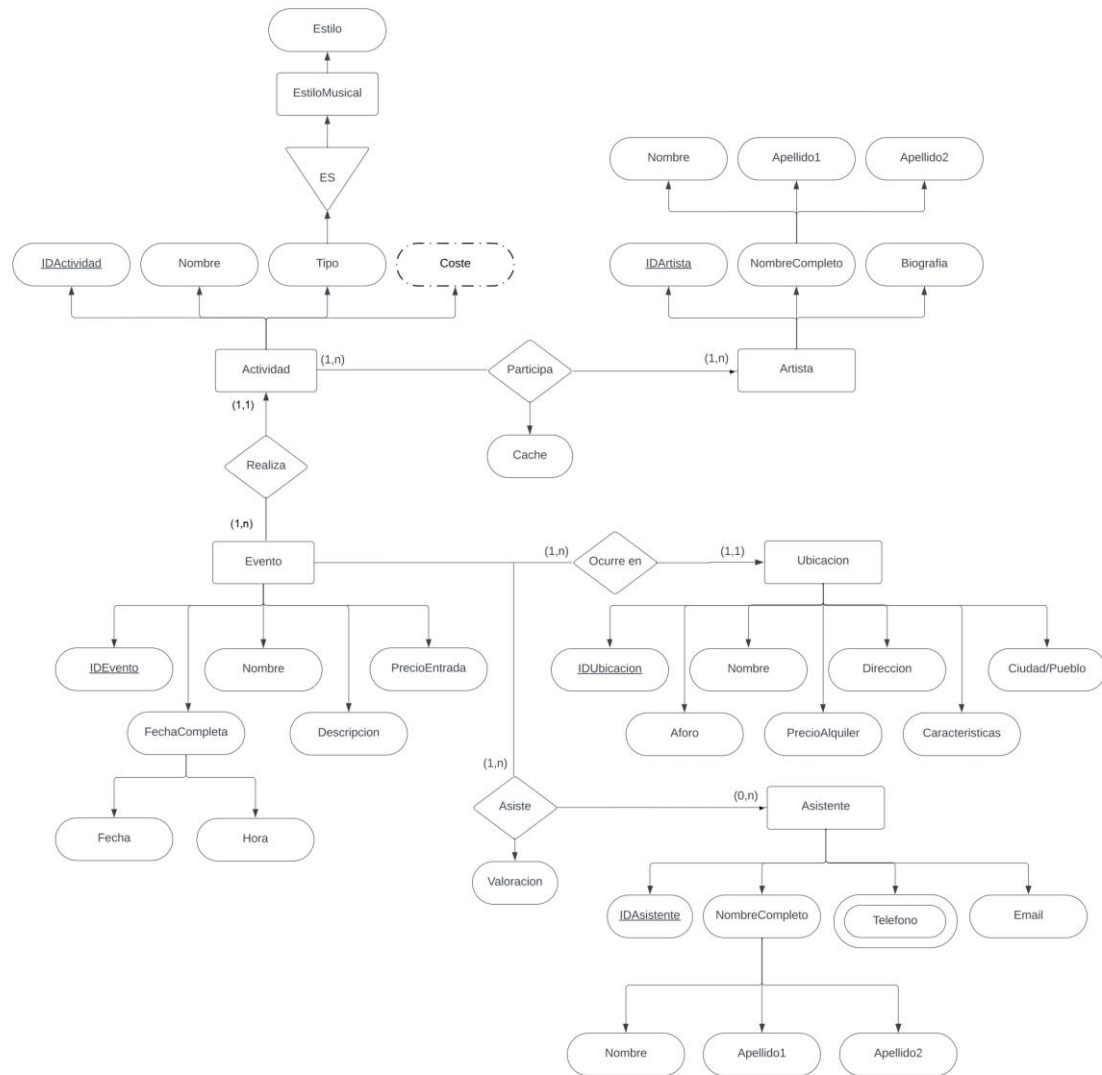


Tabla: artista

Descripción: Información de los artistas que participan en las actividades de la empresa.

Atributos:

- *idArtista* (PK): Identificador único del artista.
- *nombreCompleto*: Compuesto; nombre y apellidos como campos simples.
 - *nombre*: nombre real o artístico del artista o grupo.
 - *apellido1*: opcional, hay nombres de grupos o nombres artísticos.
 - *apellido2*: opcional, hay países que se rigen por un solo apellido.
- *biografia*: Breve descripción del artista.

Relaciones:

- **Actividad (n,n).** Un artista puede participar en varias actividades y una actividad puede contar con varios artistas.

Tabla: actividad

Descripción: Registra las distintas actividades culturales que ofrece la empresa.

Atributos:

- *idActividad* (PK): Identificador único de la actividad.
- *nombre*: Nombre de la actividad.
- *tipo*: Música, Teatro, Exposición o Conferencia. La actividad de tipo “Musica” tiene un registro en **estiloMusical** que especifica el estilo.
- *coste*: Derivado; suma del caché de los artistas que participan.

Relaciones:

- **Evento (1,n):** Una actividad puede repetirse en varios eventos, pero en un evento solo se realiza una actividad.
- **Artista (n,n):** Una actividad puede contar con varios artistas, y un artista puede participar en varias actividades.

Tabla: estiloMusical

Descripción: Es una entidad específica para las actividades de tipo “Musica”.

Tabla: ubicacion

Descripción: Contiene información detallada de los eventos (lugar, capacidad, etc.).

Atributos:

- *idUbicacion* (PK): Identificador único de la ubicación.
- *nombre*: Nombre del lugar.
- *direccion*: Dirección completa para introducir en apps como Google Maps.
- *ciudad*: Ciudad o localidad donde se encuentra.
- *aforo*: Número máximo de asistentes permitido.
- *precioAlquiler*: Precio del alquiler del lugar.
- *caracteristicas*: Detalles adicionales del lugar.

Relaciones:

- **Evento (1,n):** Una ubicación puede ser sede de distintos eventos, mientras que cada evento se realiza en una única ubicación.
-

Tabla: evento

Descripción: Contiene la información acerca de los eventos organizados por la empresa.

Atributos:

- *idEvento* (PK): Identificador único del evento.
- *nombre*: Nombre del evento.
- *idActividad* (FK → actividad.idActividad): Actividad que se realiza en el evento.
- *idUbicacion* (FK → ubicacion.idUbicacion): Ubicación del evento.
- *fechaCompleta*: Compuesto; fecha y hora como campos simples.
 - *fecha*: Informa el día, mes y año del evento
 - *hora*: Informa de la hora de inicio del evento.
- *descripcion*: Breve descripción del evento.
- *precioEntrada*: Precio de la entrada al evento.

Relaciones:

- **Actividad (1,n):** En un evento solo se realiza una actividad, pero una actividad se puede repetir en distintos eventos.
 - **Ubicacion (1,n):** Un evento se realiza en una única ubicación, pero una ubicación puede ser sede de varios eventos.
 - **Asistente** a través de la tabla **Asiste** (n,n): Un evento cuenta con múltiples asistentes, y un asistente puede acudir a distintos eventos.
-

Tabla: participa

Descripción: Relaciona a los artistas que participan en cada actividad, asignándoles un caché.

Tabla: asistente

Descripción: Registra los datos de los asistentes que compran entradas a los eventos.

Atributos:

- *idAsistente* (PK): Identificador único del asistente.
- *nombreCompleto*: Compuesto; nombre y ambos apellidos como campos simples.

- *nombre*: nombre del asistente.
- *apellido1*
- *apellido2*: Opcional, mismo motivo que en artistas.
- *email*: Correo electrónico del asistente (UNIQUE).
- *telefono*: Un asistente puede disponer de varios números de contacto. Para un uso correcto de esta información, y no violar la 1ª Forma Normal en bases de datos, se crea una tabla separada que almacena cada teléfono asociado a su asistente.

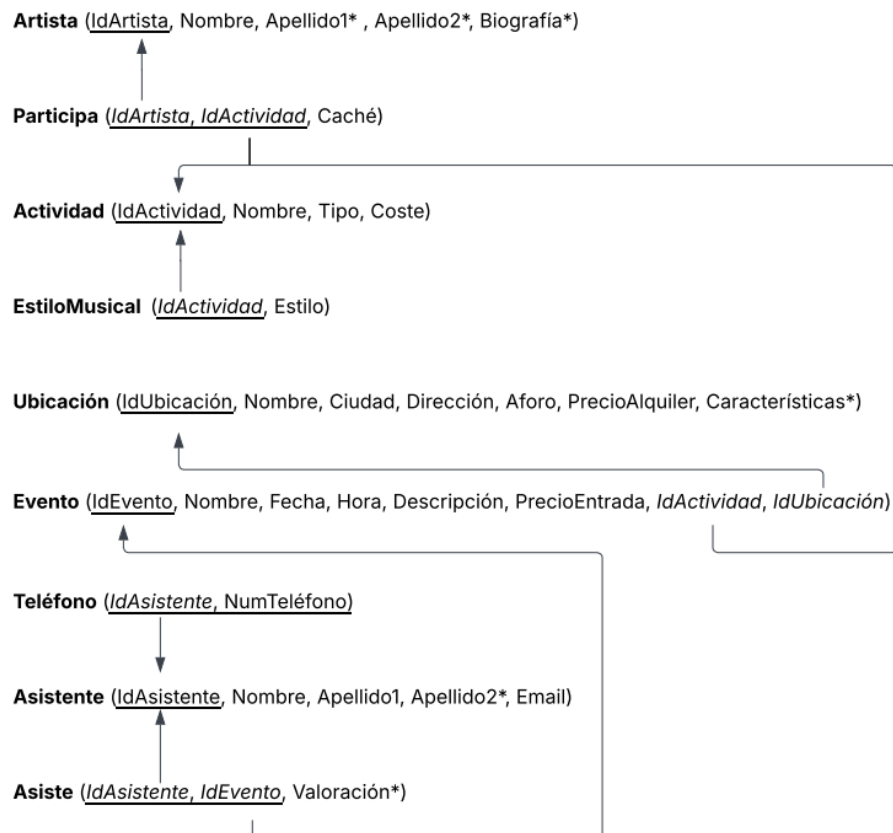
Relaciones:

- **Evento** a través de la tabla **Asiste** (n,n): Un asistente puede acudir a distintos eventos, y un evento cuenta con múltiples asistentes.

Tabla: asiste

Descripción: Registra los asistentes de cada evento, junto a una valoración (opcional) que añade el asistente sobre el evento.

2. Modelo relacional



3. Script completo copiado

```
/* -----  
Autor: Pablo Blas Hernández  
Nombre de la base de datos: artevidacultural  
----- */  
  
-- Eliminamos la base de datos si ya existe para evitar conflictos (no es obligatorio)  
DROP DATABASE IF EXISTS artevidacultural;  
  
-- Creamos la base de datos y decidimos llamarla por el nombre de la empresa  
CREATE DATABASE artevidacultural;  
USE artevidacultural;  
  
/* -----  
Definición de la estructura de la base de datos  
----- */  
  
-- Eliminamos las tablas si ya existen para evitar conflictos (no es obligatorio)  
DROP TABLE IF EXISTS artista;  
DROP TABLE IF EXISTS actividad;  
DROP TABLE IF EXISTS ubicacion;  
DROP TABLE IF EXISTS evento;  
DROP TABLE IF EXISTS asiste;  
DROP TABLE IF EXISTS telefono;  
DROP TABLE IF EXISTS asistente;  
DROP TABLE IF EXISTS participa;  
  
-- Creamos las tablas principales y derivadas que consideramos necesarias  
-- Tabla de ARTISTAS: almacena los artistas que participan en actividades  
CREATE TABLE artista (  
    idArtista INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(35) NOT NULL,  
    apellido1 VARCHAR(35), -- Puede ser el nombre artístico o de un grupo y no tener apellido
```

```
apellido2 VARCHAR(35), -- Hay países donde la gente tiene un solo apellido
biografia TEXT
);
```

-- Tabla de ACTIVIDADES: define los tipos de actividades (música, teatro, etc.) y su coste

```
CREATE TABLE actividad (
    idActividad INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(75) NOT NULL,
    tipo VARCHAR(20) NOT NULL,
    coste DECIMAL(10,2) NOT NULL DEFAULT 0
);
```

-- Tabla de ESTILOS DE MÚSICA: almacena información sobre el estilo de las actividades musicales

```
CREATE TABLE estiloMusical (
    idActividad INT PRIMARY KEY,
    estilo VARCHAR(50) NOT NULL,
    CONSTRAINT fk_tipoMusica_actividad FOREIGN KEY (idActividad)
        REFERENCES actividad(idActividad)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

-- Tabla de UBICACIONES: almacena información de los lugares donde se celebran los eventos

```
CREATE TABLE ubicacion (
    idUbicacion INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    direccion VARCHAR(100) NOT NULL,
    ciudad VARCHAR(40) NOT NULL,
    aforo INT UNSIGNED NOT NULL CHECK (aforo > 0),
    precioAlquiler DECIMAL(10,2) NOT NULL DEFAULT 0,
    características TEXT
);
```

-- Tabla de EVENTOS: incluye los datos que el cliente consultará para decidir si desea asistir o no

```
CREATE TABLE evento (  
    idEvento INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    idActividad INT NOT NULL,  
    idUbicacion INT NOT NULL,  
    fecha DATE NOT NULL,  
    hora TIME NOT NULL,  
    descripcion TEXT,  
    precioEntrada DECIMAL(10,2) NOT NULL DEFAULT 0,  
    CONSTRAINT fk_evento_actividad FOREIGN KEY (idActividad) REFERENCES  
    actividad(idActividad)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE,  
    CONSTRAINT fk_evento_ubicacion FOREIGN KEY (idUbicacion) REFERENCES  
    ubicacion(idUbicacion)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE  
);
```

-- Tabla PARTICIPA: guarda el caché (pago) del artista

```
CREATE TABLE participa (  
    idArtista INT NOT NULL,  
    idActividad INT NOT NULL,  
    cacheArtista DECIMAL(10,2) NOT NULL CHECK (cacheArtista >= 0), -- permite artistas gratis  
    PRIMARY KEY (idArtista, idActividad),  
    FOREIGN KEY (idArtista) REFERENCES artista(idArtista) ON DELETE CASCADE,  
    FOREIGN KEY (idActividad) REFERENCES actividad(idActividad) ON DELETE CASCADE  
);
```

-- Tabla ASISTENTE: contiene la información de las personas que compran entradas

```
CREATE TABLE asistente (  
    idAsistente INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(35) NOT NULL,
```



```

apellido1 VARCHAR(35) NOT NULL,

apellido2 VARCHAR(35), -- La misma razón que para artistas

email VARCHAR(75) NOT NULL UNIQUE CHECK (email REGEXP'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$') -- Asegura que el email sea normal

);

```

/* Tabla TELEFONO: almacena múltiples teléfonos para un mismo asistente (personal, de trabajo, etc.).

Si guardáramos todos los números en una sola columna, estaríamos metiendo varios datos en un solo campo, violando la 1ª Forma Normal en bases de datos. */

```

CREATE TABLE telefono (

    idAsistente INT NOT NULL,

    numTelefono VARCHAR(15) CHECK (numTelefono NOT LIKE '%[^0-9]%' AND
    LENGTH(numTelefono) BETWEEN 9 AND 15), -- Asegura que el n° de teléfono sea normal

    PRIMARY KEY (idAsistente, numTelefono),

    CONSTRAINT fk_tel_asist FOREIGN KEY (idAsistente) REFERENCES asistente(idAsistente)

    ON DELETE CASCADE

    ON UPDATE CASCADE

);

```

-- Tabla ASISTE: Guarda la valoración (de 0 a 5) que da cada asistente al evento

```

CREATE TABLE asiste (

    idAsistente INT NOT NULL,

    idEvento INT NOT NULL,

    valoracion TINYINT CHECK (valoracion BETWEEN 0 AND 5), -- no hay obligación a la valoración

    PRIMARY KEY (idAsistente, idEvento),

    CONSTRAINT fk_asist_evento FOREIGN KEY (idEvento) REFERENCES evento(idEvento)

    ON DELETE CASCADE ON UPDATE CASCADE,

    CONSTRAINT fk_asist_asistente FOREIGN KEY (idAsistente) REFERENCES asistente(idAsistente)

    ON DELETE CASCADE ON UPDATE CASCADE,

    CONSTRAINT unique_evento_asistente UNIQUE (idEvento, idAsistente)

);

```

/* -----

Trigger

Inserción de datos

```
----- */

-- TRIGGERS

DELIMITER $$

-- Trigger 1: evita superar el aforo al insertar nuevas entradas

CREATE TRIGGER control_aforo
BEFORE INSERT ON asiste
FOR EACH ROW
BEGIN
    DECLARE total_asistentes INT;
    DECLARE aforo_evento INT;

    -- Contar cuántos asistentes ya están registrados para ese evento
    SELECT COUNT(*) INTO total_asistentes
    FROM asiste
    WHERE idEvento = NEW.idEvento;

    -- Obtener el aforo máximo del evento a través de su ubicación
    SELECT u.aforo INTO aforo_evento
    FROM evento e
    JOIN ubicacion u ON e.idUbicacion = u.idUbicacion
    WHERE e.idEvento = NEW.idEvento;

    -- Si ya está lleno, lanzar error
    IF total_asistentes >= aforo_evento THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: aforo excedido. No se pueden registrar más asistentes para este evento.';
    END IF;
END$$
```

-- Trigger 1.1: controla el aforo al modificar el número de entradas

CREATE TRIGGER control_aforo_update

BEFORE UPDATE ON asiste

FOR EACH ROW

BEGIN

DECLARE total_asistentes INT;

DECLARE aforo_evento INT;

-- Solo comprobar si cambia el evento

IF NEW.idEvento <> OLD.idEvento THEN

-- Contar asistentes actuales en el nuevo evento

SELECT COUNT(*) INTO total_asistentes

FROM asiste

WHERE idEvento = NEW.idEvento;

-- Obtener aforo del evento destino

SELECT u.aforo INTO aforo_evento

FROM evento e

JOIN ubicacion u ON e.idUbicacion = u.idUbicacion

WHERE e.idEvento = NEW.idEvento;

-- Si el evento ya está lleno, impedir la actualización

IF total_asistentes >= aforo_evento THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Error: aforo excedido al intentar mover al asistente a otro evento.';

END IF;

END IF;

END\$\$

-- Trigger 2: evita duplicar eventos en la misma fecha y ubicación

CREATE TRIGGER evitar_duplicado_evento_insert

BEFORE INSERT ON evento

FOR EACH ROW

```

BEGIN

    DECLARE contador INT;

    SELECT COUNT(*) INTO contador
    FROM evento
    WHERE idUbicacion = NEW.idUbicacion
    AND fecha = NEW.fecha;

    IF contador > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: ya existe un evento en esa fecha y ubicación.';
    END IF;
END$$

```

-- Trigger 2.1: evita duplicar eventos en la misma fecha y ubicación al actualizar

```

CREATE TRIGGER evitar_duplicado_evento_update
BEFORE UPDATE ON evento
FOR EACH ROW
BEGIN
    DECLARE contador INT;

    SELECT COUNT(*) INTO contador
    FROM evento
    WHERE idUbicacion = NEW.idUbicacion
    AND fecha = NEW.fecha
    AND idEvento <> OLD.idEvento;

    IF contador > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: ya existe un evento en esa fecha y ubicación.';
    END IF;
END$$

```

DELIMITER ;

-- INSERTS (Inserción de datos)

-- Insertamos algunos artistas

INSERT INTO artista (nombre, apellido1, apellido2, biografia) VALUES

('Paula', 'Olivares', 'Iglesias', 'Soprano lirica con carrera internacional'),

('Jazz Combo ESP', NULL, NULL, 'Conjunto de jazz con 5 miembros'),

('Pablo', 'Blas', 'Hernandez', 'Pintor contemporaneo centrado en instalaciones'),

('Teatro Ensamble', NULL, NULL, 'Compania de teatro independiente'),

('Carlos', 'Rivera', 'Carnero', 'Cantante pop reconocido por su versatilidad vocal'),

('Lucia', 'Prado', 'Quintero', 'Violinista de musica clasica con trayectoria internacional'),

('Diego', 'Morales', 'Brea', 'Escultor contemporaneo especializado en materiales reciclados'),

('Elena', 'Torres', 'Gragera', 'Actriz de teatro clasico con 15 anos de experiencia'),

('Pablo', 'Rios', 'Azofra', 'Guitarrista flamenco con premios internacionales'),

('Sofia', 'Gomez', 'Urdiales', 'Pianista de jazz conocida por su estilo libre'),

('Ana', 'Catellanos', 'Noguera', 'Conferenciante en temas de arte y filosofia moderna'),

('Los Urban Blues', NULL, NULL, 'Banda de blues urbano con influencias del rock and roll'),

('Miguel', 'Hernandez', 'Lopez', 'Cantante de opera con trayectoria internacional'),

('Orquesta Sinfonica Madrid', NULL, NULL, 'Orquesta de musica clasica y contemporanea'),

('Elena', 'Martinez', 'Soler', 'Actriz y directora teatral con numerosos premios'),

('Javier', 'Sanchez', 'Ramos', 'Guitarrista de rock y fusion reconocido en Espana'),

('Laura', 'Pineda', 'Mendoza', 'Pianista de musica clasica y jazz');

-- Insertamos actividades

INSERT INTO actividad (nombre, tipo, coste) VALUES

('Concierto de Camara', 'Musica', 60.00),

('Exposicion Colectiva de Arte', 'Exposicion', 7.00),

('Obra Dramatica Horizontes', 'Teatro', 25.00),

('Concierto de Jazz', 'Musica', 30.00),

('Musical El Rey Ladron', 'Teatro', 45.00),

('Exposicion de Escultura Moderna', 'Exposicion', 7.50),

('Obra de Teatro Clasico', 'Teatro', 30.00),

('Conferencia de Filosofia y Arte', 'Conferencia', 5.00),
('Festival Pop Verano', 'Musica', 500.00),
('Recital de Guitarra Flamenca', 'Musica', 10.00),
('Festival de Blues Urbano', 'Musica', 65.00),
('Recital de Piano Clasico', 'Musica', 25.00),
('Exposicion de Fotografia Urbana', 'Exposicion', 2.50),
('Obra de Teatro Contemporaneo', 'Teatro', 30.00),
('Conferencia sobre Cine y Arte', 'Conferencia', 12.50),
('Concierto de Rock Alternativo', 'Musica', 35.00);

-- Insertamos estilos de música

INSERT INTO estiloMusical (idActividad, estilo) VALUES

(1, 'Clasica'),
(4, 'Jazz'),
(9, 'Pop'),
(10, 'Flamenco'),
(11, 'Blues'),
(12, 'Clasica'),
(16, 'Rock');

-- Insertamos ubicaciones

INSERT INTO ubicacion (nombre, direccion, ciudad, aforo, precioAlquiler, características) VALUES

('Teatro Maria Guerrero', 'C/ Principe 25', 'Madrid', 800, 1500.00, 'Teatro historico con gran acustica'),
('Sala Alternativa', 'Plaza del Arte 4', 'Barcelona', 150, 300.00, 'Espacio intimo para exposiciones'),
('Auditorio Rio', 'Av del Rio 10', 'Madrid', 2000, 5000.00, 'Auditorio con excelente acustica'),
('Centro Musical JustShow', 'Calle Tamayo 8', 'Sevilla', 800, 2500.00, 'Sala de conciertos moderna'),
('Estadio Santiago Bernabeu', 'Av Concha Espina 1', 'Madrid', 80000, 150000.00, 'Gran estadio para conciertos masivos'),
('Centro Cultural Alcobendas', 'Plaza Mayor 5', 'Barcelona', 1200, 3000.00, 'Centro municipal para eventos culturales'),
('Museo ArteVivo', 'Av Principal 32', 'Toledo', 400, 1500.00, 'Museo contemporaneo con salas de exposicion'),
('Auditorio del Mar', 'Paseo Maritimo 45', 'Malaga', 2000, 5000.00, 'Auditorio al aire libre con vista al mar'),

('Teatro Cervantes', 'C/ Malaga 12', 'Malaga', 600, 1200.00, 'Teatro historico y emblematico'),
('Galeria Urbana', 'C/ Arte 8', 'Madrid', 250, 400.00, 'Galeria para fotografia y exposiciones'),
('Auditorio Central', 'Av Cultura 45', 'Barcelona', 1500, 3500.00, 'Auditorio moderno para musica y conferencias'),
('Teatro Nuevo Sevilla', 'C/ Flamenco 3', 'Sevilla', 700, 1000.00, 'Teatro moderno para obras contemporaneas'),
('Sala Creativa', 'C/ Innovacion 7', 'Valencia', 300, 500.00, 'Espacio para conferencias y talleres culturales');

-- Insertamos eventos

INSERT INTO evento (nombre, idActividad, idUbicacion, fecha, hora, descripcion, precioEntrada)
VALUES

('VI festival de musica clasica', 1, 3, '2025-05-10', '19:30:00', 'Festival anual de musica de camara', 75.00),
('Muestra de arte contemporaneo 2025', 2, 7, '2025-06-01', '11:00:00', 'Exposicion colectiva', 8.00),
('Estreno: Horizontes', 3, 9, '2025-07-15', '20:00:00', 'Nueva produccion teatral', 25.50),
('Noche de Jazz con Jazz Combo ESP', 4, 3, '2025-08-20', '21:00:00', 'Improvisaciones en directo', 30.00),
('Musical el Rey Ladron', 5, 4, '2025-06-15', '21:00:00', 'Musical con artistas nacionales', 45.00),
('Esculturas del Futuro', 6, 7, '2025-03-20', '11:00:00', 'Exposicion de escultura reciclada', 10.00),
('Festival de Teatro Clasico', 7, 9, '2025-04-10', '19:30:00', 'Obras clasicas nacionales', 70.00),
('Charlas sobre Arte y Pensamiento', 8, 6, '2025-05-05', '17:00:00', 'Conferencia sobre arte contemporaneo', 5.00),
('Pop Verano 2025', 9, 5, '2025-07-01', '20:00:00', 'Festival pop con artistas nacionales', 60.50),
('Recital Flamenco del Sur', 10, 12, '2025-08-20', '21:00:00', 'Recital de guitarra flamenca', 12.50),
('Festival Blues Urbano', 11, 8, '2025-09-10', '22:00:00', 'Festival de blues y rock', 55.00),
('Recital Piano Clasico 2025', 12, 11, '2025-06-12', '19:00:00', 'Recital de piano clasico', 30.00),
('Exposicion Fotografia Urbana', 13, 13, '2025-07-05', '10:00:00', 'Fotografia urbana contemporanea', 5.00),
('Estreno: Teatro Contemporaneo', 14, 11, '2025-08-15', '20:30:00', 'Teatro contemporaneo en Barcelona', 22.50),
('Conferencia Cine y Arte', 15, 13, '2025-09-01', '18:00:00', 'Conferencia sobre cine y arte moderno', 7.50),
('Concierto Rock Alternativo', 16, 2, '2025-09-20', '21:00:00', 'Concierto de rock alternativo', 25.00),
('Obra experimental en Barcelona', 14, 11, '2025-10-10', '20:00:00', 'Obra teatral moderna en Barcelona', 18.00),
('Noche de Teatro en Madrid', 7, 1, '2025-11-08', '20:00:00', 'Representacion especial en el Teatro Maria Guerrero', 55.00);

-- Insertamos relaciones evento-artista

INSERT INTO participa (idArtista, idActividad, cacheArtista) VALUES

(1, 1, 35.00),
(6, 1, 50.00),
(3, 2, 20.00),
(4, 3, 40.00),
(8, 3, 25.00),
(15, 3, 30.00),
(2, 4, 25.00),
(10, 4, 30.00),
(1, 5, 100.00),
(4, 5, 60.00),
(14, 5, 55.00),
(7, 6, 35.00),
(4, 7, 45.00),
(8, 7, 25.00),
(15, 7, 50.00),
(11, 8, 12.00),
(5, 9, 50.00),
(12, 9, 65.00),
(16, 9, 40.00),
(9, 10, 20.00),
(12, 11, 100.00),
(16, 11, 75.00),
(17, 11, 45.00),
(17, 12, 50.00),
(3, 13, 20.00),
(15, 14, 70.00),
(8, 14, 35.00),
(11, 15, 25.00),
(16, 16, 65.00);

-- Insertamos asistentes


```

INSERT INTO asistente (nombre, apellido1, apellido2, email) VALUES

('Ana', 'Godoy', 'Robles', 'ana.godoy@inventado.com'),

('Luis', 'Santos', 'Orduna', 'luis.santos@inventado.com'),

('Clarie', 'Dubois', NULL, 'clarie.dubois@inventado.com'),

('Laura', 'Jimenez', 'Zamora', 'laura.jimenez@inventado.com'),

('Miguel', 'Perez', 'Bote', 'miguel.perez@inventado.com'),

('Sonia', 'Romero', 'Villajos', 'sonia.romero@inventado.com'),

('David', 'Alonso', 'Perceval', 'david.alonso@inventado.com'),

('Clara', 'Ruiz', 'Laurent', 'clara.ruiz@inventado.com'),

('Juan', 'Hernandez', 'Nogal', 'juan.hernandez@inventado.com'),

('Paula', 'Medina', 'Manzana', 'paula.medina@inventado.com'),

('William', 'Brown', NULL, 'william.brown@inventado.com'),

('Marcos', 'Gomez', 'Lopez', 'marcos.gomez@inventado.com'),

('Elena', 'Santos', 'Morales', 'elena.santos@inventado.com'),

('Ignacio', 'Ramos', 'Perez', 'ignacio.ramos@inventado.com'),

('Isabel', 'Fuentes', 'Jimenez', 'isabel.fuentes@inventado.com'),

('Ryan', 'Hoffman', NULL, 'ryan.hoffman@inventado.com');

-- Insertamos telefonos (1:N)

INSERT INTO telefono (idAsistente, numTelefono) VALUES

(1, '600111222'),

(2, '600333444'),

(3, '611222333'),

(4, '622333444'),

(5, '600555666'),

(6, '623475520'),

(7, '600222333'),

(8, '699555777'),

(9, '611888999'),

(10, '699000111'),

(11, '622123456'),

(12, '600444555'),

(13, '611777888'),

```

```
(14, '622999000'),  
(15, '675991239'),  
(16, '699123456');
```

```
-- Insertamos entradas vendidas (ASISTENCIA)
```

```
INSERT INTO asiste (idEvento, idAsistente, valoracion) VALUES
```

```
(1, 1, 5),  
(1, 2, 4),  
(4, 3, 1),  
(6, 5, 3),  
(2, 1, 4),  
(3, 1, 5),  
(4, 2, 4),  
(5, 2, NULL),  
(1, 3, 4),  
(2, 3, 5),  
(6, 4, 3),  
(7, 5, 5),  
(8, 5, 4),  
(4, 6, 4),  
(5, 6, 5),  
(3, 7, 4),  
(8, 7, 5),  
(9, 8, 5),  
(11, 9, NULL),  
(12, 10, 5),  
(13, 13, 4),  
(14, 14, 3),  
(15, 15, 2),  
(16, 16, 2),  
(17, 8, 5),  
(17, 9, 4);
```

```

/*-----
Consultas, modificaciones, borrados y vistas con enunciado
-----*/

-- VISTAS CON ENUNCIADO

-- Vista 1: Muestra el número de eventos por tipo de actividad
CREATE VIEW vista_eventos_por_tipo AS
SELECT a.tipo, COUNT(e.idEvento) AS numEventos
FROM actividad a
JOIN evento e ON a.idActividad = e.idActividad
GROUP BY a.tipo;

-- Vista 2: Calcula la recaudación total por evento
CREATE VIEW vista_recaudacion_evento AS
SELECT
    e.idEvento,
    e.nombre,
    e.fecha,
    e.precioEntrada,
    COUNT(a.idAsistente) AS entradas_vendidas,
    COUNT(a.idAsistente) * e.precioEntrada AS recaudacion
FROM evento e
LEFT JOIN asiste a ON e.idEvento = a.idEvento
GROUP BY e.idEvento;

-- Vista 3: Comprueba que no hay dos eventos en la misma fecha en la misma ubicación
CREATE VIEW vista_eventos_ocupados AS
SELECT u.nombre AS ubicacion, e.fecha, COUNT(e.idEvento) AS total_eventos
FROM evento e
JOIN ubicacion u ON e.idUbicacion = u.idUbicacion
GROUP BY u.nombre, e.fecha
HAVING total_eventos >= 1
ORDER BY u.nombre, e.fecha;

```

-- CONSULTAS

-- 1 ¿Cuál es la lista completa de eventos, especificando su tipo, ciudad y la fecha en la que tendrán lugar?

```
SELECT e.nombre AS evento, a.tipo, u.ciudad, e.fecha
FROM evento e
JOIN actividad a ON e.idActividad = a.idActividad
JOIN ubicacion u ON e.idUbicacion = u.idUbicacion
ORDER BY e.fecha ASC;
```

-- 2 ¿Cuántos eventos hay para cada tipo de actividad cultural?

```
SELECT *
FROM vista_eventos_por_tipo
ORDER BY numEventos DESC;
```

-- 3 ¿Cuáles son las dos principales ciudades que acogen más eventos?

```
SELECT u.ciudad, COUNT(e.idEvento) AS total
FROM evento e JOIN ubicacion u ON e.idUbicacion = u.idUbicacion
GROUP BY u.ciudad
ORDER BY total DESC LIMIT 2;
```

-- 4 ¿Cuántos artistas participan en cada actividad y cuáles son sus nombres?

```
SELECT
    ac.nombre AS actividad,
    COUNT(DISTINCT p.idArtista) AS num_artistas,
    GROUP_CONCAT(DISTINCT CONCAT(ar.nombre, ' ', IFNULL(ar.apellido1, ''), ' ',
    IFNULL(ar.apellido2, ''))
        ORDER BY ar.nombre SEPARATOR ', ') AS artistas
FROM actividad ac
JOIN participa p ON p.idActividad = ac.idActividad
JOIN artista ar ON p.idArtista = ar.idArtista
GROUP BY ac.idActividad, ac.nombre
ORDER BY num_artistas DESC, actividad;
```

-- 5 ¿Qué ciudades solo tienen eventos de un tipo de actividad específico?

```
SELECT
    u.ciudad,
    MAX(a.tipo) AS tipo_exclusivo
FROM ubicacion u
JOIN evento e ON e.idUbicacion = u.idUbicacion
JOIN actividad a ON e.idActividad = a.idActividad
GROUP BY u.ciudad
HAVING COUNT(DISTINCT a.tipo) = 1
    AND MAX(a.tipo) IN ('Teatro', 'Musica', 'Exposicion', 'Conferencia')
ORDER BY tipo_exclusivo, u.ciudad;
```

-- 6 ¿Cuáles son los eventos con peor valoracion media?

```
SELECT
    e.nombre,
    ROUND(AVG(a.valoracion), 2) AS promedio
FROM evento e
JOIN asiste a ON e.idEvento = a.idEvento
WHERE a.valoracion IS NOT NULL
GROUP BY e.idEvento
ORDER BY promedio ASC
LIMIT 3;
```

-- 7 ¿Cuál es la recaudación total de cada evento?

```
SELECT *
FROM vista_recaudacion_evento
ORDER BY recaudacion DESC;
```

-- 8 ¿Cuál es la recaudación media obtenida por evento organizado y por tipo de actividad?

```
SELECT
    a.tipo AS tipo_actividad,
    ROUND(AVG(v.recaudacion), 2) AS recaudacion_media
```

```

FROM vista_recaudacion_evento v
JOIN evento e ON v.idEvento = e.idEvento
JOIN actividad a ON e.idActividad = a.idActividad
GROUP BY a.tipo
UNION ALL
SELECT
    'Total' AS tipo_actividad,
    ROUND(AVG(recaudacion), 2) AS recaudacion_media
FROM vista_recaudacion_evento
ORDER BY
    CASE WHEN tipo_actividad = 'Total' THEN 0 ELSE 1 END,
    tipo_actividad;

```

-- 9 ¿Cuál es el caché de los artistas por actividad?

```

SELECT ac.idActividad,
    ac.nombre AS actividad,
    CONCAT(ar.nombre, ', ', IFNULL(ar.apellido1, ''), ', ', IFNULL(ar.apellido2, '')) AS artista,
    p.cacheArtista AS cache
FROM participa p
JOIN actividad ac ON p.idActividad = ac.idActividad
JOIN artista ar ON p.idArtista = ar.idArtista
ORDER BY ac.idActividad, p.cacheArtista DESC, artista;

```

-- 10 ¿A cuántos eventos ha asistido cada asistente y qué tipos de actividad corresponden a esos eventos?

```

SELECT
    asis.idAsistente,
    CONCAT(asis.nombre, ', ', asis.apellido1, ', ', IFNULL(asis.apellido2, '')) AS asistente,
    COUNT(e.idEvento) AS num_eventos,
    GROUP_CONCAT(DISTINCT act.tipo ORDER BY act.tipo SEPARATOR ', ') AS
tipos_actividad_asistidos
FROM asiste a
JOIN asistente asis ON a.idAsistente = asis.idAsistente
JOIN evento e ON a.idEvento = e.idEvento

```

```
JOIN actividad act ON e.idActividad = act.idActividad
```

```
GROUP BY asis.idAsistente, asistente
```

```
ORDER BY num_eventos DESC, asistente;
```

```
/* 11 ¿Cuál es la media de valoracion por tipo de actividad?
```

```
¿Cuántas valoraciones ha recibido cada tipo de actividad? */
```

```
SELECT
```

```
    ac.tipo,
```

```
    ROUND(AVG(asi.valoracion), 2) AS media_valoracion,
```

```
    COUNT(asi.valoracion) AS num_valoraciones
```

```
FROM asiste AS asi
```

```
JOIN evento AS e ON asi.idEvento = e.idEvento
```

```
JOIN actividad AS ac ON e.idActividad = ac.idActividad
```

```
WHERE asi.valoracion IS NOT NULL
```

```
GROUP BY ac.tipo
```

```
ORDER BY media_valoracion DESC;
```

```
-- 12 ¿Qué artistas participan con mayor frecuencia en las actividades?
```

```
SELECT
```

```
    CONCAT(ar.nombre, ' ', IFNULL(ar.apellido1, ''), ' ', IFNULL(ar.apellido2, '')) AS artista,
```

```
    COUNT(DISTINCT p.idActividad) AS num_actividades
```

```
FROM participa p
```

```
JOIN artista ar ON p.idArtista = ar.idArtista
```

```
GROUP BY ar.idArtista
```

```
ORDER BY num_actividades DESC, artista;
```

```
-- 13 ¿Cuáles son los días de la semana más habituales para celebrar eventos?
```

```
SELECT
```

```
    DAYNAME(e.fecha) AS dia_semana,
```

```
    COUNT(e.idEvento) AS total_eventos
```

```
FROM evento e
```

```
GROUP BY dia_semana
```

```
ORDER BY total_eventos DESC;
```

-- 14 ¿Qué asistentes han dado la mejor valoración promedio?

```
SELECT
    CONCAT(asis.nombre, ' ', asis.apellido1, ' ', IFNULL(asis.apellido2, '')) AS asistente,
    ROUND(AVG(a.valoracion), 2) AS valoracion_media,
    COUNT(a.idEvento) AS num_eventos
FROM asiste a
JOIN asistente asis ON a.idAsistente = asis.idAsistente
WHERE a.valoracion IS NOT NULL
GROUP BY asis.idAsistente, asistente
HAVING num_eventos > 0
ORDER BY valoracion_media DESC, num_eventos DESC;
```

-- 15 ¿Hay algún evento programado para el día x en la ubicación x?

```
SELECT *
FROM vista_eventos_ocupados
WHERE ubicacion = 'Teatro Maria Guerrero' -- Cambiar por la ubicación que quieras comprobar
AND fecha = '2025-11-08'; -- Cambiar por la fecha que quieras comprobar
```